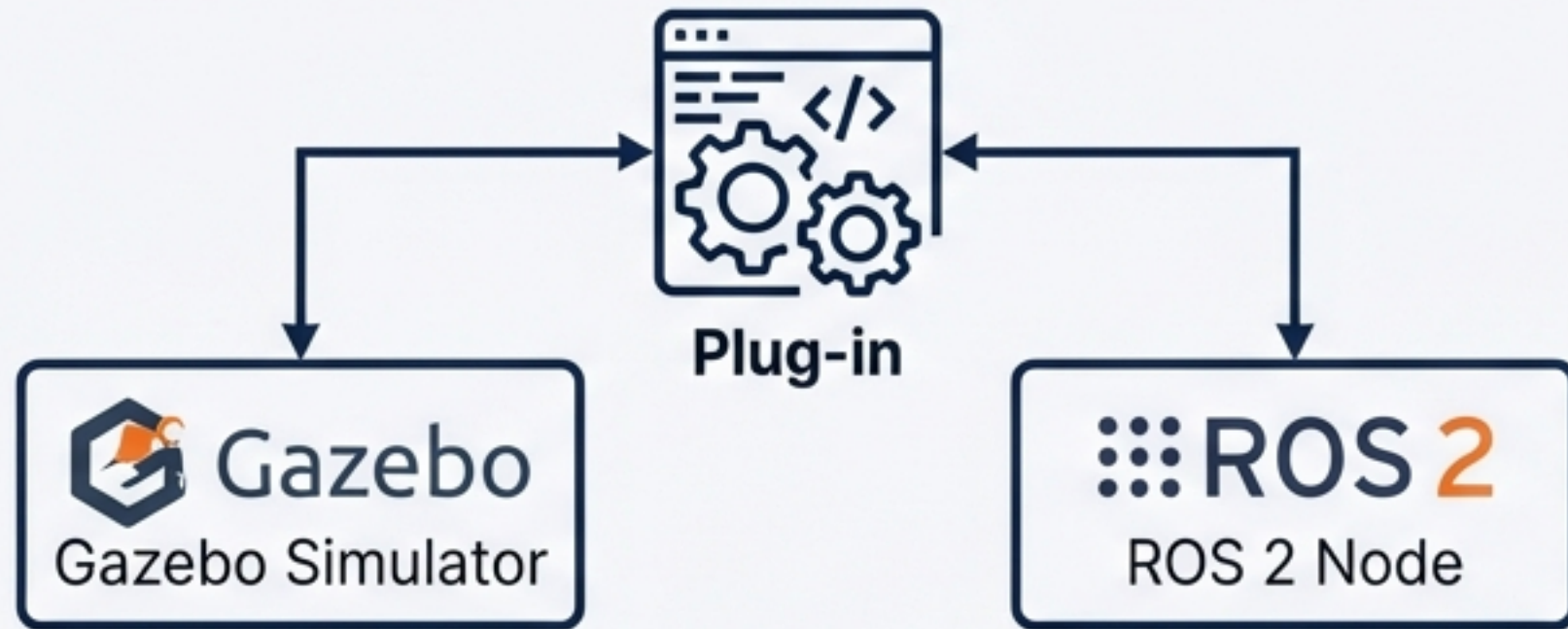


Percobaan 10: Konsep Dasar World Plugin C++



Konsep Utama: Menggunakan **libmy_world_plugin.so** untuk memperluas kapabilitas Gazebo via SDF tag **<plugin>**.

```
ros2 launch gazebo_praktikum  
percobaan10_advanced.launch.py
```

Pro-Tip



Set Environment Variable di dalam launch file untuk mengarahkan GAZEBO_PLUGIN_PATH ke folder lib hasil instalasi package (ament_index), memastikan path selalu benar.

Implementasi C++: Anatomi my_world_plugin.cpp

Struktur Kelas: Inherits dari gazebo::WorldPlugin

```
#include <gazebo/gazebo.hh>
#include <gazebo_ros/node.hpp>
#include <std_msgs/msg/float64.hpp>

namespace gazebo
{
  class MyWorldPlugin : public WorldPlugin
  {
  public:
    // (1) FUNGSI LOAD
    void Load(physics::WorldPtr _world, sdf::ElementPtr _sdf) override
    {
      // (2) INISIALISASI NODE ROS 2
      ros_node_ = gazebo_ros::Node::Get(_sdf);

      // (3) SETUP PUBLISHER
      pub_ = ros_node_->create_publisher<std_msgs::msg::Float64>(
        "/plugin/sim_time", 10);

      // (4) KONEKSI KE WORLD UPDATE EVENT
      update_connection = event::Events::ConnectWorldUpdateBegin(
        std::bind(&MyWorldPlugin::OnUpdate, this));

      RCLCPP_INFO(ros_node_>get_logger(), "My World Plugin Loaded!");
    }
  private:
    void OnUpdate()
    {
      // Implementasi update callback di sini
    }

    gazebo_ros::Node::SharedPtr ros_node_;
    rclcpp::Publisher<std_msgs::msg::Float64>::SharedPtr pub_;
    event::ConnectionPtr update_connection_;
  };

  GZ_REGISTER_WORLD_PLUGIN(MyWorldPlugin)
}
```

1 Load(WorldPtr, sdf::ElementPtr):
Dipanggil otomatis saat Gazebo memuat world.

2 gazebo_ros::Node::Get(sdf):
Mengambil rclcpp node untuk komunikasi ROS 2.

3 std_msgs::msg::Float64:
Publisher waktu simulasi ke topik /plugin/sim_time.

4 Events::ConnectWorldUpdateBegin(callback):
Melakukan sinkronisasi dengan physics step Gazebo pada frekuensi tinggi (~1000 Hz).

Arsitektur Build: Konfigurasi CMakeLists.txt

find_package: Wajib memuat gazebo REQUIRED dan gazebo_ros REQUIRED.

```
cmake_minimum_required(VERSION 3.8)
project(my_world_plugin_pkg)

# find_package: Wajib memuat gazebo REQUIRED dan gazebo_ros REQUIRED.
find_package(ament_cmake REQUIRED)
find_package(gazebo REQUIRED)
find_package(gazebo_ros REQUIRED)
find_package(rclcpp REQUIRED)
find_package(std_msgs REQUIRED)

# add_library: Membuat shared library dari src/my_world_plugin.cpp.
add_library(my_world_plugin SHARED
  src/my_world_plugin.cpp
)

# ament_target_dependencies: Menautkan rclcpp, gazebo_ros, dan std_msgs.
ament_target_dependencies(my_world_plugin
  rclcpp
  gazebo_ros
  std_msgs
)

# install: Arahkan install(TARGETS my_world_plugin DESTINATION lib) agar
# terinstal dan dapat ditemukan oleh ekosistem Gazebo.
install(TARGETS my_world_plugin
  DESTINATION lib
)

ament_package()
```

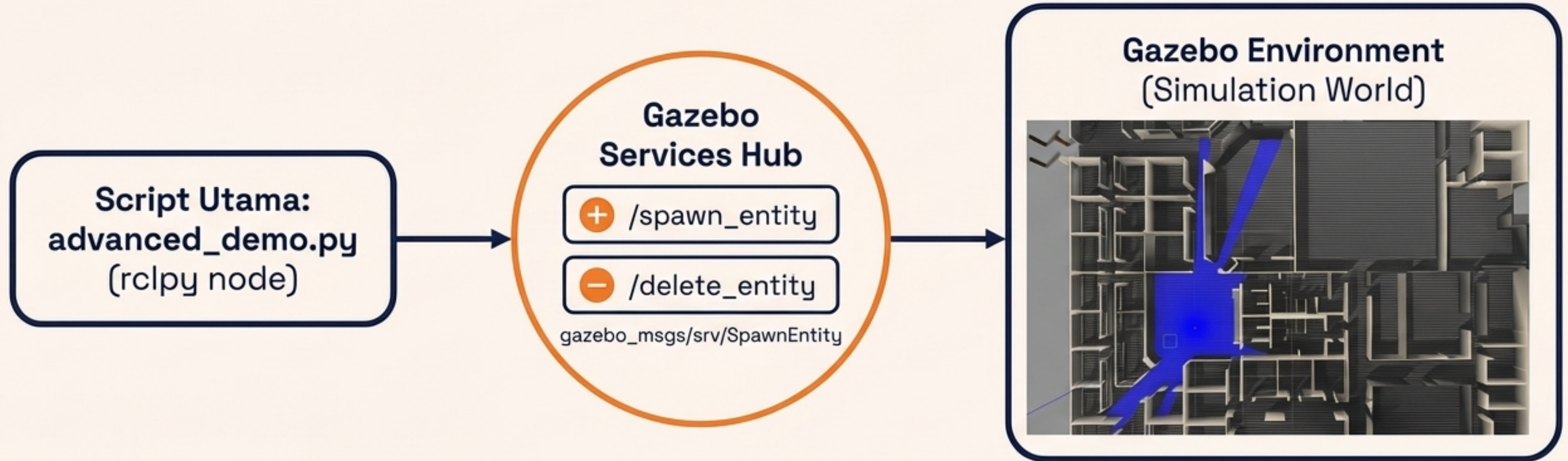
add_library:
Membuat shared library dari src/my_world_plugin.cpp.

ament_target_dependencies:
Menautkan rclcpp, gazebo_ros, dan std_msgs.

install:
Arahkan install(TARGETS my_world_plugin DESTINATION lib) agar terinstal dan dapat ditemukan oleh ekosistem Gazebo.

Note: Tambahkan juga GAZEBO_INCLUDE_DIRS dan GAZEBO_LIBRARIES untuk direktori target.

Dynamic Spawn via Gazebo Service

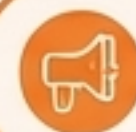


Field Data Wajib:

- **name:** String unik (identitas objek).
- **xml:** String SDF model.
- **initial_pose:** geometry_msgs/Pose.

Skenario Penggunaan:

- Memunculkan rintangan dinamis secara acak.
- Target objek yang berpindah tempat.
- Multi-stage experiment.



Contoh Eksekusi: Spawn objek box berukuran 0.3m di koordinat (2, 0, 0.5).

Monitoring Output Lanjutan & Utilitas Plugin

```
ros2 topic echo /plugin/status (Tipe String: status world)
```

```
ros2 topic echo /plugin/sim_time (Tipe Float64: waktu simulasi)  
ros2 topic hz /plugin/sim_time (Pengukuran frekuensi physics update, target ~1000 Hz)
```



Automated Logging:
Pencatatan data eksperimen otomatis.



Event Triggering:
Memicu event berdasarkan batas parameter simulasi.



Custom Virtual Sensors: Integrasi sensor non-standar.

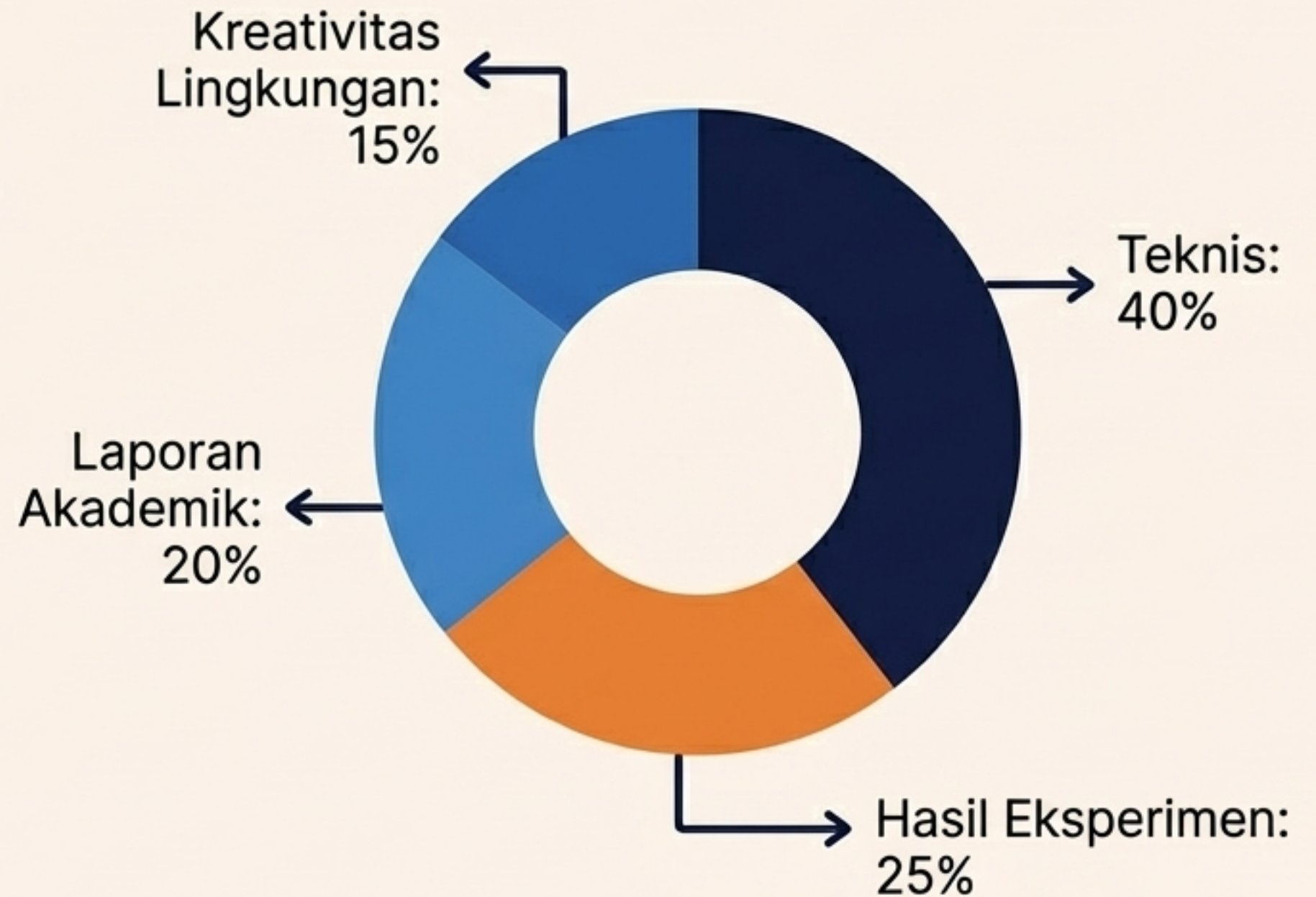


Runtime Physics Control: Modifikasi parameter fisika (seperti gravitasi/gesekan) secara real-time.

Capstone Project: Simulasi Robot Otonom

Simulasi robot otonom di lingkungan kustom (Kelompok: 2-3 mahasiswa).

- ✓ 1. ROS 2 Package Lengkap (World + URDF/XACRO + Launch).
- ✓ 2. Peta Lingkungan Kustom (SLAM .yaml & .pgm).
- ✓ 3. Demo Navigasi Otonom (Minimal 2 waypoint).
- ✓ 4. Laporan Teknis PDF (8+ Halaman).
- ✓ 5. Video Demonstrasi (10-15 Menit).



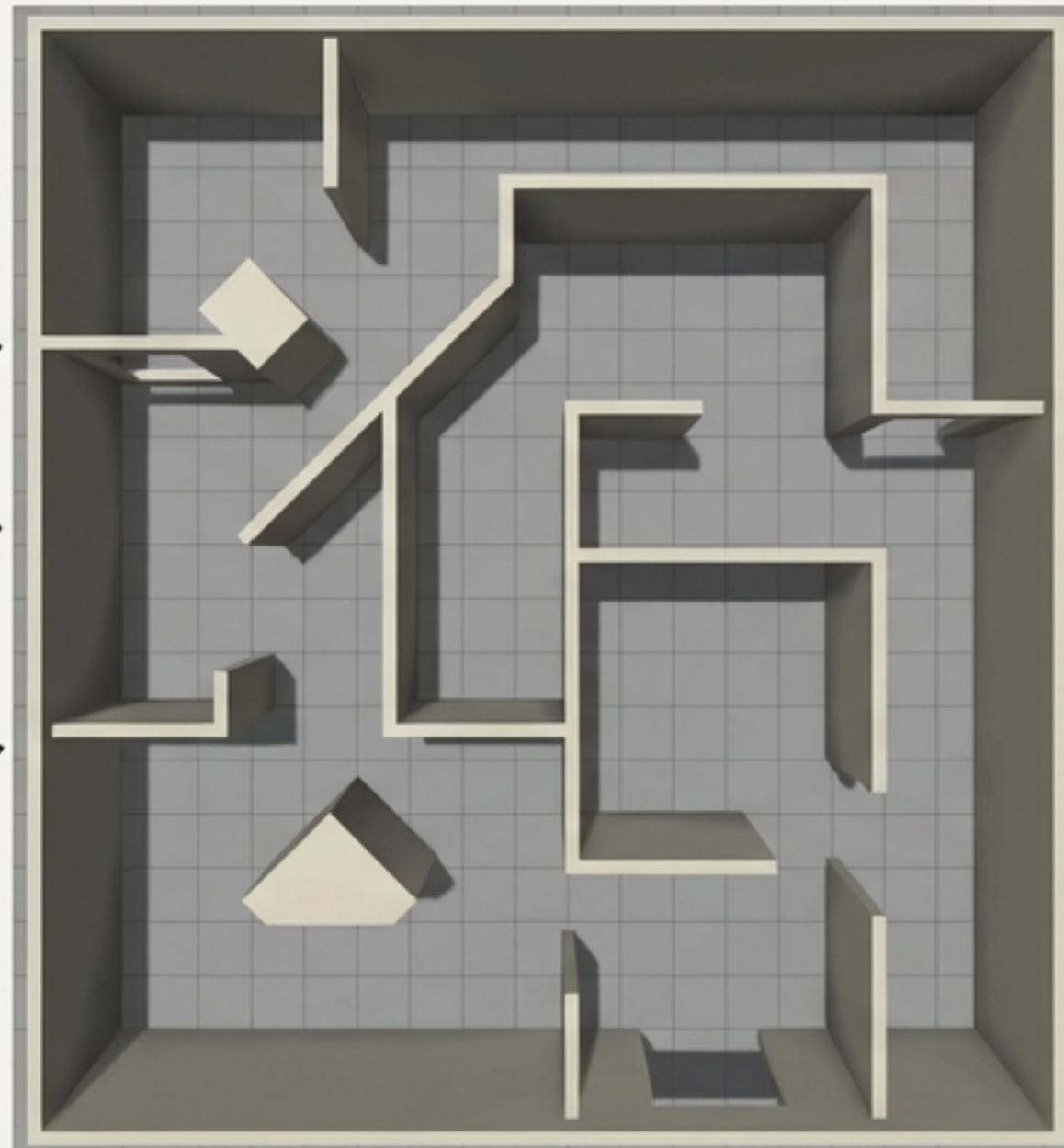
Project Fase 1: Desain World Kustom



1 Ruang berukuran minimal 10x10 meter.

Dibatasi oleh 4 dinding tertutup.

Memiliki 3 rintangan dengan bentuk berbeda di dalamnya.



Workflow Desain

- Gunakan alat Gazebo Building Editor (via menu menu Edit) atau tulis SDF XML secara manual.
- Wajib tambahkan pencahayaan directional dan tekstur material lantai.

```
worlds/my_world.world>  
- Simpan file akhir pada: worlds/my_world.world.  
</property>
```

Verifikasi

- Test menggunakan template launch file Percobaan 1, ubah argumen world file. Ambil screenshot sebagai bukti di laporan.

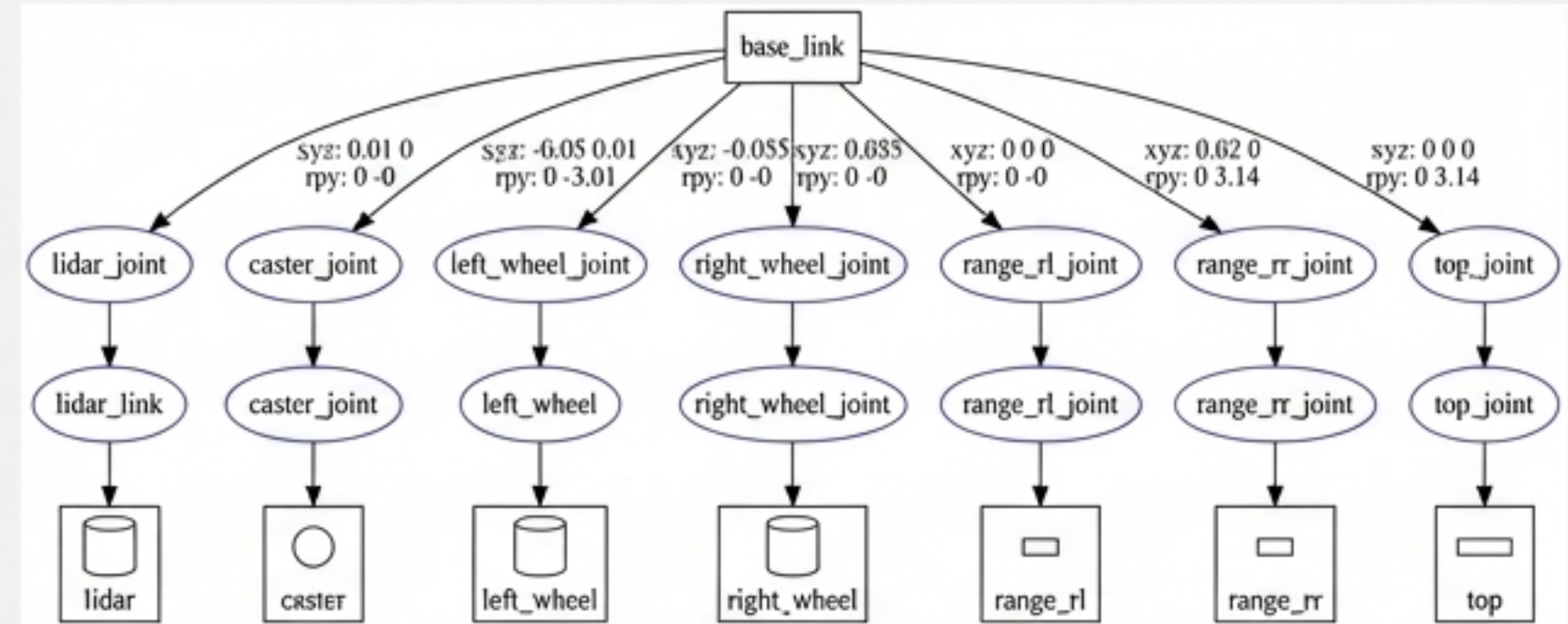
```
-launch launch file Percobaan 1 \  
  ambil Percobaan 1 --v "worlds/my_world.world"
```


Project Fase 2: URDF Robot Kustom



- **Komponen Robot (robot_kustom.urdf.xacro):**

- **base_link:** Box geometry (0.3 x 0.2 x 0.1 m).
- **Roda:** 2 roda continuous joint (radius 0.05 m, separation 0.2 m).
- **Caster:** Fixed joint sphere (radius 0.025 m).
- **lidar_link:** Sensor LIDAR (Cylinder).



- **Gazebo Plugins Wajib:**

diff_drive plugin & ray_sensor plugin.

Validasi & Visualisasi:

- Cek Sintaks: `xacro robot.xacro | check_urdf`.
- Gunakan template Percobaan 3 untuk tes visual di RViz2. Sertakan diagram link-joint ke dalam laporan.

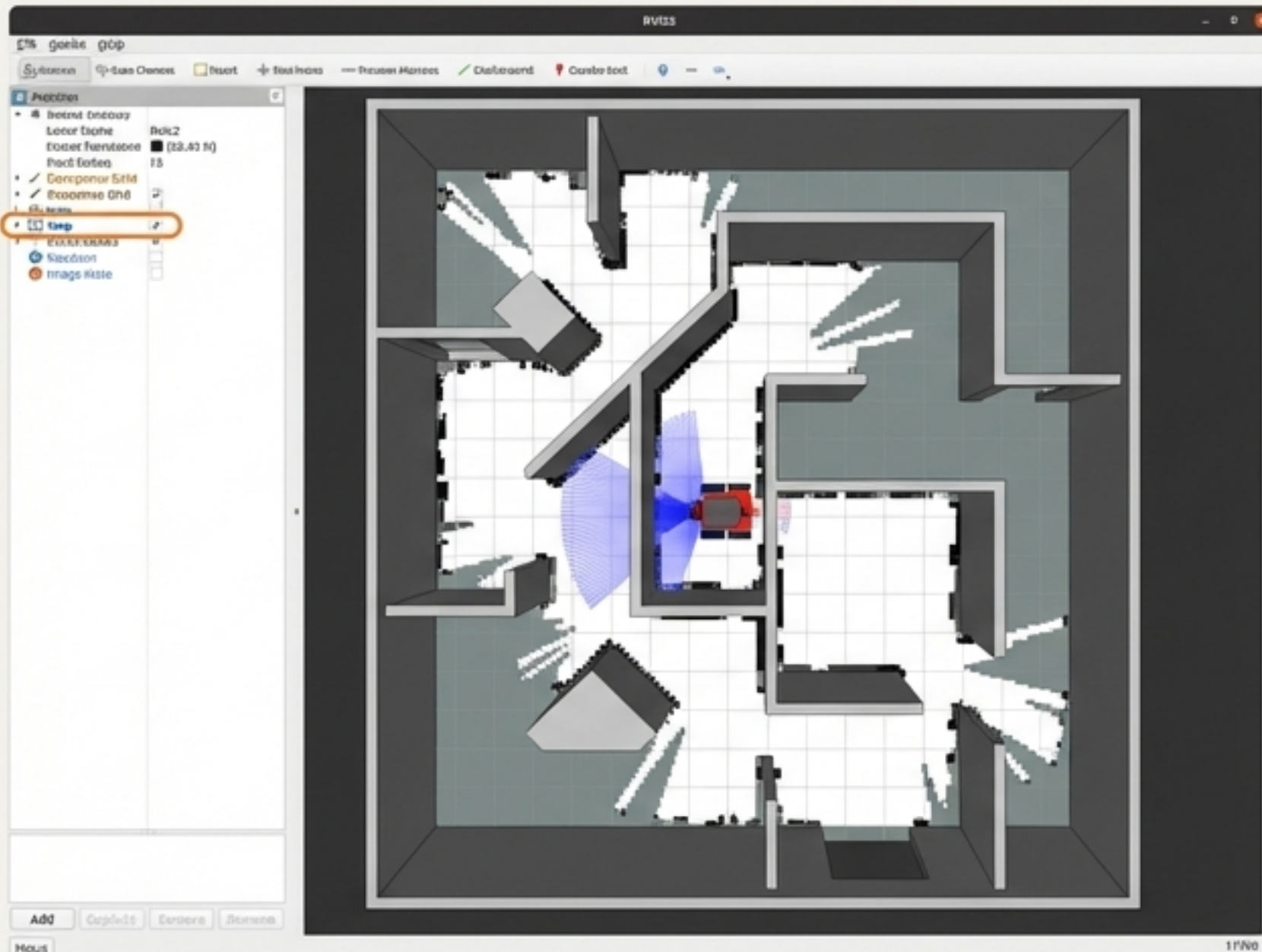
Project Fase 3: SLAM & Pemetaan Lingkungan

Fase 1
Gazebo

Fase 2
Gazebo-blue

Fase 3
Gazebo-blue

Fase 4



Prosedur SLAM

- Gunakan percobaan7A sebagai template base, override file world dan XACRO dengan model kustom.
- Jalankan node teleop pada terminal terpisah. Eksplorasi ruangan hingga mendapatkan coverage peta yang solid.



Monitoring RViz2

- Fokus pada panel Map (Pastikan Durability diatur ke Transient Local).

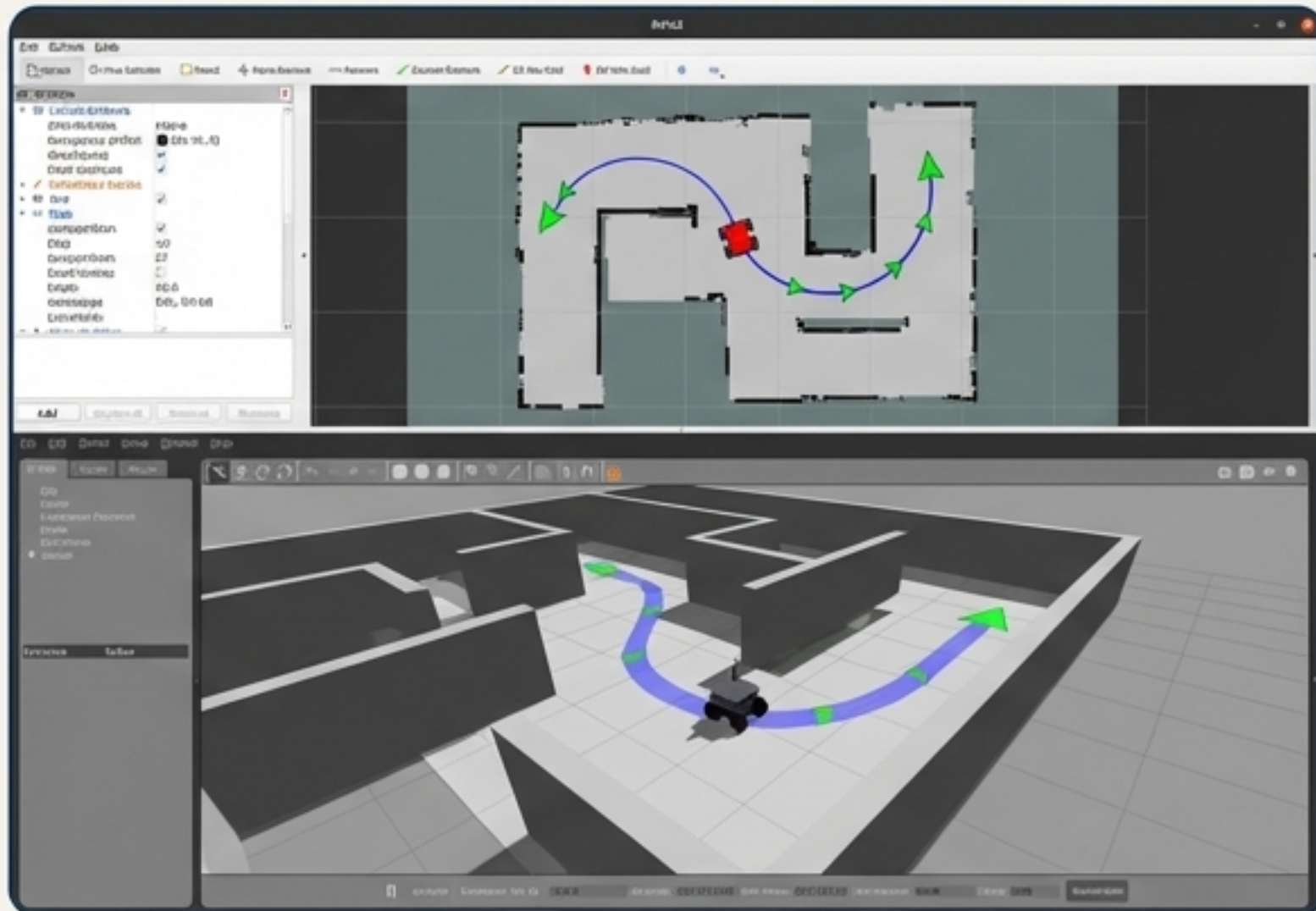
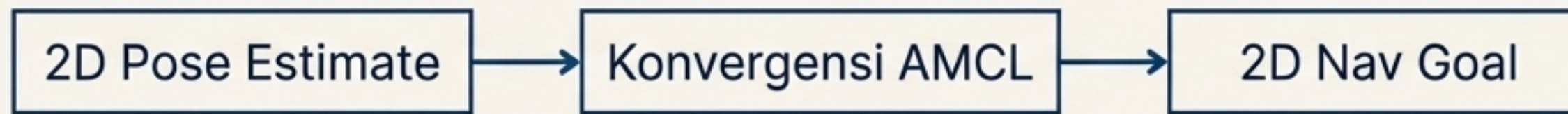


Simpan Peta Lingkungan

```
ros2 run nav2_map_server map_saver_cli -f  
~/ros2_ws/maps/nama_peta
```

- Verifikasi eksistensi file .yaml (metadata) dan .pgm (grid gambar). Masukkan screenshot peta ke laporan.

Project Fase 4: Navigasi Otonom (Nav2)



Inisialisasi Nav2:

Launch menggunakan template `percobaan7B` dengan argument `map:=/path/my_map.yaml`.

Prosedur Operasional:

1. Set koordinat inisial di RViz2 agar partikel AMCL konvergen dengan posisi spawn Gazebo.
2. Setel target waypoint menggunakan 2D Nav Goal. Tunggu robot tiba secara otonom dan hindari obstacle.
3. Setel target kedua setelah konfirmasi tiba di titik pertama.

 **Tuning:** Lakukan eksperimen modifikasi parameter Nav2 (seperti kecepatan dan jarak inflation) untuk optimalisasi pergerakan. Rekam video manuver tersebut.

Dokumentasi Teknis: Struktur Laporan

Format Dokumen: PDF, minimum 8 halaman.

Bab 1:
Pendahuluan &
Objektif Praktikum.

Bab 2: Desain
Lingkungan Kustom
(Lampirkan visual
world).

Bab 3: Arsitektur
URDF (Sertakan
diagram link-joint).

Bab 4: Konfigurasi
SLAM & Hasil
Ekstraksi Peta
(.pgm view).

Bab 5: Analisis
Parameter Nav2
(nav2_params.yaml).

Bab 6: Hasil,
Manuver Path
Plan, Diskusi
(Kendala & Solusi).

Bab 7:
Kesimpulan
Eksperimen.

Bab 8: Referensi
(ROS 2, Nav2,
slam_toolbox).

Pengingat Penilaian: Teknis 40%, Hasil 25%, Kreativitas 15%, Laporan 20%.

Panduan Teknis: Tugas Video Demomonstrasi



Durasi & Format: 10 - 15 menit, mencakup 10 percobaan + Demo Project Akhir.

Persyaratan Produksi:

- - **Alat Perekam:** Gunakan OBS Studio atau Kazam (Screen recorder).
- - **Layout Video:** Screen record jelas untuk RViz2/Gazebo/Terminal, Webcam overlay wajah di pojok layar.
- - **Audio:** Narasi deskriptif yang jelas dalam Bahasa Indonesia.

Struktur Waktu (Garis Besar):

-  1. Intro (Konsep ROS 2): 1 Menit
-  2. P1-P5 (Dasar): 3 Menit
-  3. P6-P7A (SLAM): 3 Menit
-  4. P7B-P10 (Nav2/Multi-Robot/Plugin): 4 Menit
-  5. Demo Project: 3 Menit
-  6. Kesimpulan: 1 Menit

Storyboard Video: Bagian 1 & 2 (Percobaan 1-5)



Membuka Gazebo (Empty World). Menjatuhkan objek primitif shape, menunjukkan demonstrasi fisika gravitasi dan collision.

Peluncuran URDF tanpa Gazebo. Gerakkan slider pada joint_state_publisher_gui dan observasi pergerakan link di RViz2 secara real-time.

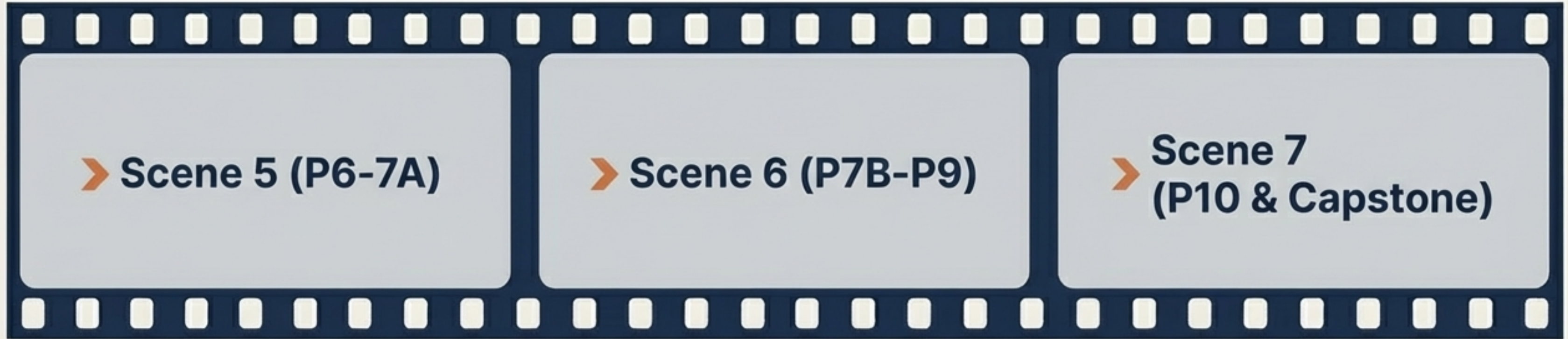
Menyorot fitur asinkron: robot spawn di dalam Gazebo secara presisi dengan delay 2 detik setelah simulator terbuka.

Sensor check. Mengaktifkan panel Camera dan Laser LaserScan di RViz2. Mengetik perintah "

```
ros2 topic echo /scan --no-arr
```

di terminal untuk menunjukkan keberhasilan aliran data LIDAR.

Storyboard Video: Bagian 3 & 4 (Advanced & Project)



Dual terminal teleop & proses pemetaan.
Membuka terminal pembantu, menunjukkan odometri di **RViz2** yang memvisualisasikan peta labirin bertahap.

Nav2 otonom dengan penghindaran rintangan.
Transisi ke lengan manipulator 3-DOF bergerak presisi.
Simulasi multi-robot di mana dua namespace dikendalikan via terminal independen.

Echo [/plugin/sim_time](#) untuk pembuktian plugin.
Terakhir, demonstrasi 3 menit dari Lingkungan Kustom, **Navigasi Peta Baru**, dan interaksi **2 target waypoint otonom**.

Kesimpulan & Referensi Modul 04

✓ Pencapaian Modul:

Mahasiswa telah menguasai seluruh spektrum dari Empty World, integrasi URDF, Sensor, Teleop, SLAM, Nav2, Manipulator, Namespace Multi-Robot, hingga Custom C++ Plugin.

Tautan Referensi Penting

- 🔗 - ROS 2 Humble Docs: docs.ros.org/en/humble
- 🔗 - Gazebo Classic: classic.gazebosim.org
- 🔗 - Nav2 Stack: navigation.ros.org
- 🔗 - SLAM Toolbox: github.com/SteveMacenski/slam_toolbox
- 🔗 - ros2_control: control.ros.org

👁️ Sneak Peek

Selanjutnya: Bersiap menuju Modul 05 – Computer Vision dan Persepsi Robot (Robot Perception).